```
print "{$lcname} > " . lcfirst($lcname) . PHP_EOL;
print "{$lcname} > " . mb_lcfirst($lcname) . PHP_EOL;
print "{$ucname} > " . mb_ucfirst($ucname);
```

```
Output:
łukasz > łukasz
łukasz > łukasz
ŁUKASZ > ŁUKASZ
```

The first line of output shows that the core function `lcfirst()` has no effect (because the first byte of character U+0142 is *0xC5*, outside the ASCII Latin-1 range). The next lines show the `mbstring` functions doing their thing, converting the UTF-8 Polish characters correctly. Bicameral character sets (having upper and lowercase forms) will convert correctly, but with a small caveat:

```
print mb_ucfirst("idil"); // Correct transform should be İdil - U+0130
```

```
Output:
Idil
```

This looks like we have a regional problem. For the Turkish character `i`, the correct uppercase form is İ, but `mb_ucfirst()` converts it to a standard Latin capital `I`. Unicode has a set of special rules for these localization issues, so perhaps we just need to change the locale before we make the function call?

```
setlocale(LC_ALL, 'tr_TR.UTF-8');
print mb_ucfirst("idil"); // Not working despite locale
```

```
Output:
Idil
```

It still doesn't work. The reason for this is that both `mb_ucfirst()` and `mb_lcfirst()` are internally calling another function, `mb_convert_case()`, and the "problem" (it's a deliberate design choice) lies there.

mb_convert_case()

`strtoupper()`, `ucfirst()`, and all the rest do what they do and are fine for straightforward use-cases. `mb_convert_case()`, on the other hand, offers all of the utility of the functions we've seen in this section by specifying different modes (via function argument) and, of course, offering multi-byte support. Many of the `mbstring` functions